

VLLM专题（三十四）—vLLM 分页注意力机制（Paged Attention）



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

目前，vLLM 使用了自己实现的多头查询注意力内核（[csrc/attention/attention_kernels.cu](https://github.com/vllm-project/vllm/blob/main/csrc/attention/attention_kernels.cu)）。该内核设计为与 vLLM 的分页 KV 缓存兼容，其中键（key）和值（value）缓存存储在单独的块中（注意，这里的“块”概念与 GPU 线程块不同。因此，在后续文档中，我将 vLLM 分页注意力块称为“块”，而将 GPU 线程块称为“线程块”）。

为了实现高性能，该内核依赖于一种特殊设计的内存布局和访问方法，特别是在线程将数据从全局内存读取到共享内存时。本文档的目的是逐步提供内核实现的高层次解释，帮助那些希望了解 vLLM 多头查询注意力内核的人。通过阅读本文档，用户可能会更好地理解内核实现，并更容易跟进实际代码。

请注意，本文档可能不会涵盖所有细节，例如如何计算对应数据的正确索引或点积（dot multiplication）的实现。然而，在阅读本文档并熟悉高层次逻辑流程后，您应该能够更容易地阅读实际代码并理解细节。

1. 输入

内核函数接收一系列参数，供当前线程执行其分配的任务。其中最重要的三个参数是输入指针 `q`、`k_cache` 和 `v_cache`，它们指向全局内存中需要读取和处理的查询（query）、键（key）和值（value）数据。输出指针 `out` 指向全局内存中结果应写入的位置。这四个指针实际上引用的是多维数组，但每个线程仅访问分配给它的部分数据。为了简化，这里省略了所有其他运行时参数。

```
template<
typename scalar_t,
int HEAD_SIZE,
int BLOCK_SIZE,
int NUM_THREADS,
int PARTITION_SIZE = 0>
__device__ void paged_attention_kernel(
... // Other side args.
const scalar_t* __restrict__ out,    // [num_seqs, num_heads, max_num_partitions, head_size]
const scalar_t* __restrict__ q,      // [num_seqs, num_heads, head_size]
const scalar_t* __restrict__ k_cache, // [num_blocks, num_kv_heads, head_size/x, block_size, x]
const scalar_t* __restrict__ v_cache, // [num_blocks, num_kv_heads, head_size, block_size]
... // Other side args.
)
```

在函数签名上方，还有一系列在编译时确定的模板参数。

- `scalar_t` 表示查询（query）、键（key）和值（value）数据元素的数据类型，例如 FP16。
- `HEAD_SIZE` 表示每个头（head）中的元素数量。
- `BLOCK_SIZE` 表示每个块（block）中的 token 数量。
- `NUM_THREADS` 表示每个线程块（thread block）中的线程数量。
- `PARTITION_SIZE` 表示张量并行（tensor parallel）GPU 的数量（为了简化，我们假设其为 0，即禁用张量并行）。

有了这些参数，我们需要执行一系列准备工作。这包括计算当前的头索引（head index）、块索引（block index）以及其他必要的变量。然而，目前我们可以暂时忽略这些准备工作，直接进入实际计算部分。一旦我们掌握了整个流程，这些准备工作会更容易理解。

2. 概念

在深入计算流程之前，我想先介绍一些后续章节中会用到的概念。如果你在阅读过程中遇到任何不熟悉的术语，可以先跳过本节，稍后再回来查阅。

序列（Sequence）：

序列代表一个客户端请求。例如，`q` 指向的数据形状为 `[num_seqs, num_heads, head_size]`，这表示 `q` 指向了总共 `num_seqs` 个查询序列数据。由于这是一个单查询注意力内核，每个序列只有一个查询 token。因此，`num_seqs` 等于批次中处理的总 token 数量。

上下文（Context）：

上下文由序列生成的 token 组成。例如，`["What", "is", "your"]` 是上下文 token，而输入的查询 token 是 `"name"`。模型可能会生成 token `"?"`。

向量（Vec）：

向量是一组一起获取和计算的元素。对于查询和键数据，向量大小（`VEC_SIZE`）被确定，以便每个线程组可以一次获取和计算 16 字节的数据。对于值数据，向量大小（`V_VEC_SIZE`）被确定，以便每个线程可以一次获取和计算 16 字节的数据。例如，如果 `scalar_t` 是 FP16（2 字节）且 `THREAD_GROUP_SIZE` 为 2，则 `VEC_SIZE` 为 4，而 `V_VEC_SIZE` 为 8。

线程组（Thread group）：

线程组是一个小型的线程组（`THREAD_GROUP_SIZE`），它一次获取和计算一个查询 token 和一个键 token。每个线程只处理 token 数据的一部分。一个线程组处理的总元素数称为 `x`。例如，如果线程组包含 2 个线程且头大小为 8，则线程 0 处理索引为 0、2、4、6 的查询和键元素，而线程 1 处理索引为 1、3、5、7 的元素。

线程 1 处理索引为 1、5、9、13 的元素。

块 (Block) :

vLLM 中的键和值缓存数据被分割成块。每个块存储固定数量 (`BLOCK_SIZE`) 的 token 数据, 且每个块只包含一个头的数据。每个块可能只包含整个上下文 token 的一部分。例如, 如果块大小为 16 且头大小为 128, 则对于一个头, 一个块可以存储 $16 * 128 = 2048$ 个元素。

Warp:

Warp 是一组 32 个线程 (`WARP_SIZE`), 它们在流式多处理器 (SM) 上同时执行。在本内核中, 每个 warp 一次处理一个查询 token 与一个完整块的键 token 之间的计算 (它可能会在多次迭代中处理多个块)。例如, 如果有 4 个 warp 和一个上下文的 6 个块, 则分配方式可能是: warp 0 处理第 0 和第 4 块, warp 1 处理第 1 和第 5 块, warp 2 处理第 2 块, warp 3 处理第 3 块。

线程块 (Thread block) :

线程块是一组线程 (`NUM_THREADS`), 它们可以访问相同的共享内存。每个线程块包含多个 warp (`NUM_WARPS`), 在本内核中, 每个线程块处理一个查询 token 与整个上下文的键 token 之间的计算。

网格 (Grid) :

网格是线程块的集合, 并定义了集合的形状。在本内核中, 形状为 (`num_heads, num_seqs, max_num_partitions`)。因此, 每个线程块只处理一个头、一个序列和一个分区的计算。

3. 查询 (Query)

本节将介绍查询数据在内存中的存储方式以及每个线程如何获取这些数据。如上所述, 每个线程组获取一个查询 token 的数据, 而每个线程本身只处理一个查询 token 数据的一部分。在每个 warp 内, 每个线程组都会获取相同的查询 token 数据, 但会将其与不同的键 token 数据相乘。

```
const scalar_t* q_ptr = q + seq_idx * q_stride + head_idx * HEAD_SIZE;
```

每个线程定义了它自己的 `q_ptr`，该指针指向全局内存中分配给它的查询 token 数据。例如，如果 `VEC_SIZE` 为 4 且 `HEAD_SIZE` 为 128，那么 `q_ptr` 指向的数据包含总共 128 个元素，这些元素被划分为 $128 / 4 = 32$ 个向量（vecs）。

```
__shared__ Q_vec q_vecs[THREAD_GROUP_SIZE][NUM_VECS_PER_THREAD];
```

接下来，我们需要将 `q_ptr` 指向的全局内存数据读取到共享内存中，作为 `q_vecs`。需要注意的是，每个 `vecs` 被分配到不同的行。例如，如果 `THREAD_GROUP_SIZE` 为 2，线程 0 将处理第 0 行的 `vecs`，而线程 1 则处理第 1 行的 `vecs`。通过这种方式读取查询数据，相邻的线程（如线程 0 和线程 1）可以读取相邻的内存，从而实现内存合并，以提高性能。

4. 键（Key）

与“查询（Query）”部分类似，本节介绍了键的内存布局和分配方式。虽然每个线程组在一次内核运行中只处理一个查询（query）token，但它可能会在多次迭代中处理多个键（key）token。同时，每个 warp 将在多次迭代中处理多个键 token 块，确保在内核运行结束后，所有的上下文 token 都能被整个线程组处理完毕。在这里，“处理”指的是在查询数据和键数据之间执行点积运算。

```
const scalar_t* k_ptr = k_cache + physical_block_number * kv_block_stride
    + kv_head_idx * kv_head_stride
    + physical_block_offset * x;
```

与 `q_ptr` 不同，每个线程中的 `k_ptr` 在不同迭代中会指向不同的键（key）token。如上所述，`k_ptr` 指向的键 token 数据是基于 `k_cache` 中分配的块（block）、分配的头（head）以及分配的 token 来确定的。

上图展示了键（key）数据的内存布局。假设 `BLOCK_SIZE` 为 16，`HEAD_SIZE` 为 128，`x` 为 8，`THREAD_GROUP_SIZE` 为 2，并且总共有 4 个 warps。每个矩形代表一个头（head）中一个键 token 的所有元素，这些元素将由一个线程组处理。左半部分展示了 warp 0 的总共 16 个键 token 数据块，而右半部分则代表其他 warps 或迭代中剩余的键 token 数据。在每个矩形内部，总共有 32 个 `vecs`（一个 token 的 128 个元素），这些 `vecs` 将由 2 个线程（一个线程组）分别处理。

```
K_vec k_vecs[NUM_VECS_PER_THREAD]
```

接下来，我们需要从 `k_ptr` 中读取键（key）token 数据，并将它们存储到寄存器内存中作为 `k_vecs`。我们使用寄存器内存来存储 `k_vecs`，因为它只会被一个线程访问一次，而 `q_vecs` 会被多个线程多次访问。每个 `k_vecs` 将包含多个向量，用于后续的计算。每个 `vec` 将在每次内部迭代中设置。`vecs` 的分配方式使得 warp 中的相邻线程可以一起读取相邻的内存，从而再次促进内存合并。例如，线程 0 将读取 `vec 0`，而线程 1 将读取 `vec 1`。在下一个内部循环中，线程 0 将读取 `vec 2`，线程 1 将读取 `vec 3`，依此类推。

你可能对整个流程仍然有些困惑。别担心，请继续阅读接下来的“QK”部分。它将以一种更清晰、更高层次的方式说明查询（query）和键（key）的计算流程。

5. QK

如下面的伪代码所示，在整个 `for` 循环块之前，我们获取一个 token 的查询（query）数据并将其存储在 `q_vecs` 中。接着，在外部 `for` 循环中，我们遍历指向不同 token 的 `k_ptrs`，并在内部 `for` 循环中准备 `k_vecs`。最后，我们在 `q_vecs` 和每个 `k_vecs` 之间执行点积运算。

```

q_vecs = ...
for ... {

    k_ptr = ...
    for ... {

        k_vecs[i] = ...
    }

    ...

    float qk = scale * Qk_dot<scalar_t, THREAD_GROUP_SIZE>::dot(q_vecs[thread_group_offset], k_vecs);
}

```

如前所述，对于每个线程，它一次只获取部分查询（query）和键（key）token 数据。然而，在 `Qk_dot<>::dot` 中会发生跨线程组的归约操作。因此，这里返回的 `qk` 不仅仅是部分查询和键 token 点积的结果，而是整个查询和键 token 数据之间的完整结果。

例如，如果 `HEAD_SIZE` 的值为 128，且 `THREAD_GROUP_SIZE` 为 2，那么每个线程的 `k_vecs` 将包含总共 64 个元素。然而，返回的 `qk` 实际上是 128 个查询元素和 128 个键元素之间点积的结果。如果你想了解更多关于点积和归约的细节，可以参考 `Qk_dot<>::dot` 的实现。不过，为了简洁起见，本文档将不涉及这部分内容。

6. Softmax

接下来，我们需要为所有 `qks` 计算归一化的 softmax，如上图所示，其中每个 x 代表一个 `qk`。为此，我们必须获取 `qk_max` ($m(x)$) 的归约值以及所有 `qks` 的 `exp_sum` ($\ell(x)$)。归约操作应在整个线程块范围内执行，涵盖查询 (query) token 与所有上下文键 (key) token 之间的结果。

$$\begin{aligned} m(x) &:= \max_i x_i \\ f(x) &:= \left[e^{x_1 - m(x)} \sum_i \dots e^{x_B - m(x)} \right] \\ \ell(x) &:= \sum_i f(x)_i \\ \text{softmax}(x) &:= \frac{f(x)}{\ell(x)} \end{aligned}$$

6.1 qk_max 和 logits

在获取 `qk` 结果后，我们可以立即用 `qk` 设置临时的 `logits` 结果（最终，`logits` 应存储归一化的 softmax 结果）。同时，我们可以比较并收集当前线程组计算的所有 `qks` 的 `qk_max`。

```
if (thread_group_offset == 0) {  
  
    const bool mask = token_idx >= context_len;  
    logits[token_idx - start_token_idx] = mask ? 0.f : qk;  
    qk_max = mask ? qk_max : fmaxf(qk_max, qk);  
}
```

请注意，这里的 `logits` 位于共享内存中，因此每个线程组将为其分配的上下文 token 设置相应的字段。总体而言，`logits` 的大小应为上下文 token 的数量。


```
for (int mask = WARP_SIZE / 2; mask >= THREAD_GROUP_SIZE; mask /= 2) {

    qk_max = fmaxf(qk_max, VLLM_SHFL_XOR_SYNC(qk_max, mask));

}

if (lane == 0) {

    red_smem[warp_idx] = qk_max;

}
```

接下来，我们需要在每个 warp 内获取归约后的 `qk_max`。主要思想是让 warp 中的线程相互通信，以获取最终的 `qk_max`。

```
for (int mask = NUM_WARPS / 2; mask >= 1; mask /= 2) {

    qk_max = fmaxf(qk_max, VLLM_SHFL_XOR_SYNC(qk_max, mask));

}

qk_max = VLLM_SHFL_SYNC(qk_max, 0);
```

最后，我们可以通过比较该线程块中所有 warps 的 `qk_max`，获取整个线程块的归约 `qk_max`。然后，我们需要将最终结果广播到每个线程。

6.2 exp_sum

与 `qk_max` 类似，我们也需要从整个线程块中获取归约后的 `exp_sum` 值。

```

for (int i = thread_idx; i < num_tokens; i += NUM_THREADS) {

    float val = __expf(logits[i] - qk_max);
    logits[i] = val;
    exp_sum += val;
}
...
exp_sum = block_sum<NUM_WARPS>(&red_smem[NUM_WARPS], exp_sum);

```

首先，对每个线程组的 `exp` 值求和，同时将 `logits` 中的每个条目从 `qk` 转换为 `exp(qk - qk_max)`。请注意，这里的 `qk_max` 已经是整个线程块中的最大 `qk`。然后，我们可以像处理 `qk_max` 一样，对整个线程块的 `exp_sum` 进行归约。

```

const float inv_sum = __fdividef(1.f, exp_sum + 1e-6f);
for (int i = thread_idx; i < num_tokens; i += NUM_THREADS) {

    logits[i] *= inv_sum;
}

```

最后，通过归约后的 `qk_max` 和 `exp_sum`，我们可以得到最终的归一化 softmax 结果，即 `logits`。这个 `logits` 变量将在后续步骤中用于与值（value）数据进行点积运算。此时，它应存储所有分配的上下文 token 的 `qk` 的归一化 softmax 结果。

7. 值

现在我们需要提取值（value）数据并与 `logits` 执行点积运算。与查询（query）和键（key）不同，值数据没有线程组（thread group）的概念。如图所示，与键 token 的内存布局不同，值数据中同一列的元素对应于同一个值 token。对于一块值数据，它有 `HEAD_SIZE` 行和 `BLOCK_SIZE` 列，这些数据被分割成多个 `v_vecs`。

每个线程每次总是从相同的 `V_VEC_SIZE` 个 token 中提取 `V_VEC_SIZE` 个元素。因此，单个线程通过多次内部迭代从不同行但相同列中提取多个 `v_vecs`。对于每个 `v_vec`，它需要与相应的 `logits_vec` 进行点积运算，`logits_vec` 也是从 `logits` 中提取的 `V_VEC_SIZE` 个元素。总体而言，通过多次内部迭代，每个 warp 将处理一个值 token 块。而通过多次外部迭代，整个上下文的价值 token 将被处理完毕。

```
float accs[NUM_ROWS_PER_THREAD];
for ... {
    // Iteration over different blocks.
    logits_vec = ...
    for ... {
        // Iteration over different rows.
        v_vec = ...
        ...
        accs[i] += dot(logits_vec, v_vec);
    }
}
```

如上伪代码所示，在外部循环中，与 `k_ptr` 类似，`logits_vec` 会遍历不同的块，并从 `logits` 中读取 `V_VEC_SIZE` 个元素。在内部循环中，每个线程从相同的 token 中读取 `V_VEC_SIZE` 个元素作为 `v_vec`，并执行点积运算。需要注意的是，在每次内部迭代中，线程会为相同的 token 提取不同头位置（head position）的元素。点积结果随后会累加到 `accs` 中。因此，`accs` 的每个条目都映射到当前线程分配的头位置。

例如，如果 `BLOCK_SIZE` 为 16，`V_VEC_SIZE` 为 8，则每个线程每次会为 8 个 token 提取 8 个值元素。每个元素来自不同 token 的同一头位置。如果 `HEAD_SIZE` 为 128，`WARP_SIZE` 为 32，那么在每次内部循环中，一个 warp 需要提取 `WARP_SIZE * V_VEC_SIZE = 256` 个元素。这意味着一个 warp 需要 8 次内部迭代来处理整个值 token 块。每个线程中的 `accs` 包含 8 个元素，这些元素在 8 个不同的头位置上累加。对于线程 0，`accs` 变量将包含 8 个元素，这些元素是值头（value head）的第 0、32、...、224 个元素，它们是从所有分配的 8 个 token 中累加得到的。

8. LV

现在，我们需要在每个 warp 内对 `accs` 进行归约操作。这一过程使得每个线程能够为分配的头位置（head positions）累加一个块中所有 token 的 `accs`。

```
for (int i = 0; i < NUM_ROWS_PER_THREAD; i++) {  
  
    float acc = accs[i];  
    for (int mask = NUM_V_VECS_PER_ROW / 2; mask >= 1; mask /= 2) {  
  
        acc += VLLM_SHFL_XOR_SYNC(acc, mask);  
    }  
    accs[i] = acc;  
}
```

接下来，我们对所有 warps 的 `accs` 进行归约操作，使得每个线程能够为分配的头位置（head positions）累加所有上下文 token 的 `accs`。需要注意的是，每个线程中的 `accs` 仅存储了整个头（head）中部分元素的累加结果，这些结果来自所有上下文 token。然而，总体而言，输出的所有结果都已经计算完成，只是存储在不同的线程寄存器内存中。

```

float* out_smem = reinterpret_cast<float*>(shared_mem);
for (int i = NUM_WARPS; i > 1; i /= 2) {

    // Upper warps write to shared memory.
    ...

    float* dst = &out_smem[(warp_idx - mid) * HEAD_SIZE];
    for (int i = 0; i < NUM_ROWS_PER_THREAD; i++) {

        ...

        dst[row_idx] = accs[i];
    }

    // Lower warps update the output.
    const float* src = &out_smem[warp_idx * HEAD_SIZE];
    for (int i = 0; i < NUM_ROWS_PER_THREAD; i++) {

        ...

        accs[i] += src[row_idx];
    }

    // Write out the accs.
}

```

9. 输出（Output）

现在，我们可以将所有从本地寄存器内存中计算出的结果写入最终的输出全局内存中。

```

scalar_t* out_ptr = out + seq_idx * num_heads * max_num_partitions * HEAD_SIZE
    + head_idx * max_num_partitions * HEAD_SIZE
    + partition_idx * HEAD_SIZE;

```

首先，我们需要定义 `out_ptr` 变量，它指向分配的序列（sequence）和分配的头（head）的起始地址。

```
for (int i = 0; i < NUM_ROWS_PER_THREAD; i++) {  
  
    const int row_idx = lane / NUM_V_VECS_PER_ROW + i * NUM_ROWS_PER_ITER;  
    if (row_idx < HEAD_SIZE && lane % NUM_V_VECS_PER_ROW == 0) {  
  
        from_float(*(out_ptr + row_idx), accs[i]);  
    }  
}
```

最后，我们需要遍历不同的分配头位置，并根据 `out_ptr` 写出相应的累加结果。